

## Sequences and Tuples

Sequence: A sequence objects is a list of these objects in some order.

### Condition:

1. Order
2. first parentheses.

Example: 1.  $(7, 21, 57)$  sequence

2.  $(7, 21, 14)$  not

Infinite sequence: "..." Continue the seq forever.

$(7, 21, 57, \dots)$ ,  $(1, 5, 10, 15, \dots)$

### Finite sequence:

$(1, 5, 10, 15)$

Tuple: finite sequence are called Tuples.

### Sequence condition:

1. Order
2. first parentheses
3. finite sequence

Example:

①  $(1, 5, 10, 15) \rightarrow 4\text{-tuple}$

②  $(1, 5) \rightarrow 2\text{-tuple} \rightarrow \text{Ordered pair}$

Unordered pair  $\rightarrow$  set  $\{ \}$

Ordered pair  $\rightarrow$  Tuple  $\langle \rangle$

Cross Product

If  $A$  and  $B$  are two sets, the cartesian Product / Cross product of  $A$  and  $B$ , written  $A \times B$ , is the set of all ordered pairs where in the first element is a member of  $A$  and second element is a member of  $B$ .

Example:

$$A = \{1, 2\} \text{ \& } B = \{a, b, c\}$$

$$A \times B = \{(1, a), (1, b), (1, c), (2, a), (2, b), (2, c)\}$$

## Functions

A function is an object that sets up an input-output relationship.

Example:  $f(x) = x^2$

$\downarrow$   $\rightarrow$

input output

$=$  Domain = Range

$$x = 2, y = 4$$

$$x = 3, y = 9$$

$$x = 4, y = 16$$

Domain:  $(-\infty, \infty)$

Range:  $[0, \infty)$



## Strings and Language

Alphabet: we define an alphabet to be any non-empty finite set.

$$\Sigma_1 = \{a, b\} \quad \Gamma_1 = \{1, 2\}$$

$$\Sigma_2 = \{b, c\}$$

String: A string over an alphabet is a finite sequence of symbols from that alphabet.

### Example

$$\Sigma = \{a, b\}$$

①  $a, b$

②  $aa, bb, ab, ba$

③  $aaa, aab, aba, baa,$   
 $abb, bab, bba, bbb$

④  $\text{length} = 0 \quad \epsilon$



## Strings and Language

Alphabet: we define an alphabet to be any non-empty finite set.

$$\Sigma_1 = \{a, b\} \quad \Gamma_1 = \{1, 2\}$$

$$\Sigma_2 = \{b, c\}$$

String: A string over an alphabet is a finite sequence of symbols from that alphabet.

### Example

$$\Sigma = \{a, b\}$$

①  $a, b$

②  $aa, bb, ab, ba$

③  $aaa, aab, aba, baa, abb, bab, bba, bbb$

④  $\text{length} = 0 \quad \epsilon$

Language: Collection of all possible strings.

$$\Sigma = \{a, b\}$$

Example

$L_1 = \{ \text{string of length '3'} \}$

$= \{ aaa, aab, aba, baa, abb, bab, bba, bbb \}$

Finite language

$L_2 = \{ \text{string of length '2'} \}$

$= \{ aa, ab, ba, bb \}$

Infinite language

$L_3 = \{ \text{string starting with 'a'} \}$

$= \{ a, aa, aaa, ab, aab, aba, \dots \}$

## Power of $\Sigma$

$$\Sigma = \{a, b\}$$

$$\Sigma^0 = \text{string of length '0'} = \{\epsilon\}$$

$$\Sigma^1 = \text{string of length '1'} = \{a, b\}$$

$$\Sigma^2 = \text{string of length '2'} = \{aa, ab, ba, bb\}$$

$$\Sigma \cdot \Sigma = \{a, b\} \{a, b\}$$

$$= \{aa, ab, ba, bb\}$$

$$\Sigma^3 = \text{string of length '3'} = \{aaa, aab, aba, abb, baa, bab, bba, bbb\}$$

Kleene closure

$$\Sigma^* = 2^n \rightarrow \text{Infinite possibilities}$$

$$\Sigma^+ = \text{Positive closure}$$

$$\Sigma^* = \Sigma^0 + \Sigma^+$$

$$\Sigma^+ = \Sigma^* - \Sigma^0$$



## Definitions

Definitions describes the objects and notions that we use of a term.

Precision is essential to any mathematical definition.

Example: A prime numbers is a natural number greater than one that has no positive divisors other than one and the number itself.

2, 3, 4, 5, 6, 7, ...

## Theorem

A theorem is a mathematical statement on the basis of previously established statements.

Example: There is infinite numbers of prime numbers.

## Proofs

A proof is a convincing logical argument that a statement is true.

Example: finite (N numbers of prime numbers).

$P_1, P_2, P_3, P_4, \dots, P_N$

$K = P_1 \times P_2 \times P_3 \times \dots \times P_N + 1$

↓

Prime number  $n > 1$

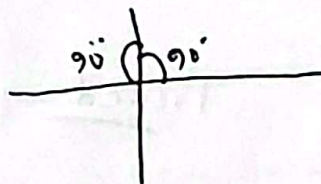
Infinite,

## Lemmas

A minor result whose sole purpose is to help in proving a theorem.

### Example:

Theorem 1: 

Theorem 2: 

$$n = a + a = 180^\circ$$

$$= 2a = 180^\circ$$

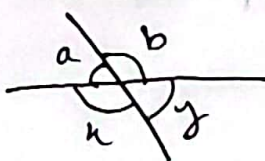
$$\therefore a = 90^\circ$$

## Corollaries

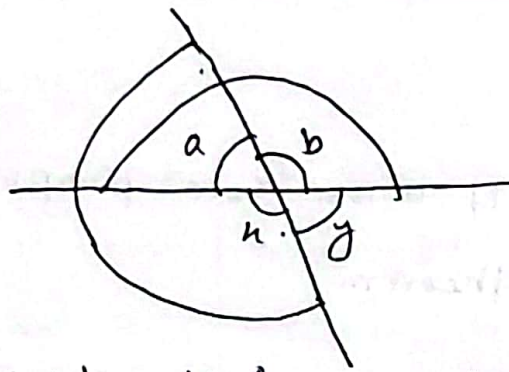
A result in which the proof relies heavily on a given theorem.

### Example:

Theorem 1: 

Theorem 2: 

Prove that,  $a = y$   
 $b = x$



$$a + b = 180^\circ \Rightarrow b = 180^\circ - a$$

$$a + h = 180^\circ \Rightarrow h = 180^\circ - a$$

$$\therefore b = h$$

Proved

How to prove a theorem

- ① Identify the assumptions and goals of the theorem. উদ্ভা
- ② Understand the implications of each of the assumptions made. ফলাফল
- ③ Make a assumption about what we are trying to prove and show that it leads to a proof or contradiction.



## Type of Proof

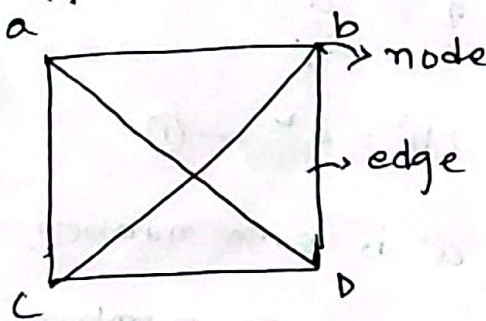
- ① Proof by construction
- ② Proof by Contradiction
- ③ Proof by Induction

### ① Proof by Construction

- A particular type object exist
- How to construct the object (one way)

Theorem: for each numbers  $n$  greater than 2, there exists a 3-regular graph with  $n$  nodes.

$k$ -regular graph: we define a graph to be  $k$ -regular if every node in the graph has degree  $k$ .

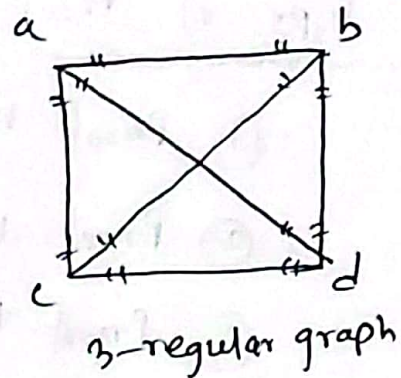


3-regular graph

degree(a) = 3  
degree(b) = 3  
degree(c) = 3  
degree(d) = 3

Condition:

- ① Even number
- ②  $n \geq 2$  ( $n = \text{node}$ )



Proof by Contradiction

We assume that the theorem is false and show that this assumption leads to an obviously false consequence called a contradiction.

Theorem:  $\sqrt{2}$  is irrational

Proof:  $\sqrt{2}$  is rational

Let,  $\sqrt{2} = \frac{a}{b}$ ; [ $a$  and  $b$  are integers]

$$\Rightarrow 2 = \frac{a^2}{b^2}$$

$$\Rightarrow 2b^2 = a^2 \quad \text{--- ①}$$

$a^2$  is even number.

$\therefore a$  is even number.

Condition

①  $a$  and  $b$  co-prime  
i.e. no common factor  
between  $a$  and  $b$ .

② Irreducible  
 $a = 5, b = 6; \frac{5}{6}$

$$a = 2k : [k \text{ is an integer}]$$

from eqn ①,

$$2b^2 = (2k)^2$$

$$\Rightarrow 2b^2 = 4k^2$$

$$\Rightarrow b^2 = 2k^2 : [k \text{ is an integer}]$$

$b^2$  is even number

$b$  is an even number.

So,  $\sqrt{2}$  is irrational

[Proved]

### ③ Proof by Induction

Proof by induction is an advanced method used to show that all elements of an infinite set have a specified property.

Base  $\rightarrow t = 0$

Induction step  $\rightarrow t = k$

$t = k+1$



### Question:

base case  $\rightarrow P_0 = P$  is the amount of loan

Amount of loan after one month  $\rightarrow P_1 = MP_0 - Y$

Amount of loan after two month  $\rightarrow P_2 = MP_1 - Y$

show that, for each  $t \geq 0$ ,  $P_t = PM^t - Y \left( \frac{M^t - 1}{M - 1} \right)$

### Proof:

base case: for  $t=0$ ,  $P_0 = PM^0 - Y \left( \frac{M^0 - 1}{M - 1} \right)$

$$\Rightarrow P_0 = P.$$

Induction step: for each  $k \geq 0$ ,  $P_k = PM^k - Y \left( \frac{M^k - 1}{M - 1} \right)$

we have to show that,  $P_{k+1} = PM^{k+1} - Y \left( \frac{M^{k+1} - 1}{M - 1} \right)$

Here,

$$P_{k+1} = MP_k - Y$$

$$= M \left[ PM^k - Y \left( \frac{M^k - 1}{M - 1} \right) \right] - Y$$

$$= PM^{k+1} - Y \left( \frac{M^{k+1} - M}{M - 1} \right) - Y \left( \frac{M - 1}{M - 1} \right)$$

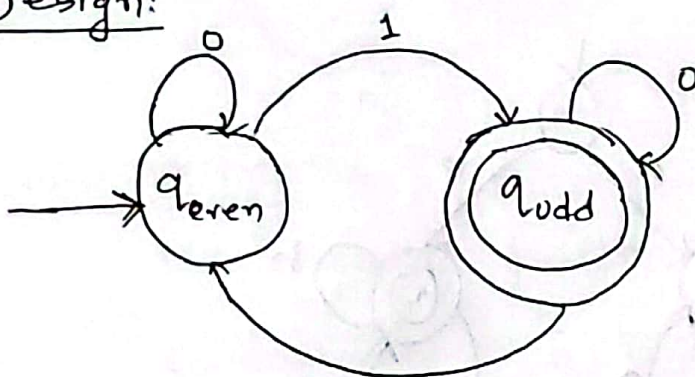
$$\Rightarrow P_{k+1} = PM^{k+1} - Y \left( \frac{M^{k+1} - 1}{M - 1} \right)$$

| Proved |

\* Construct a finite automaton to recognize the language consists of all strings with an odd number of 1s. where  $\Sigma = \{0, 1\}$

$= \{1, 111, 010, 001011011, \dots\}$

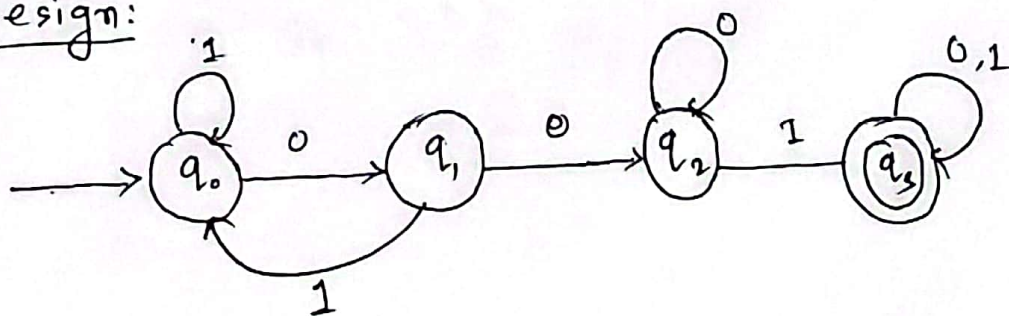
Design:



\* Design a finite automaton to recognize the regular language of all strings that contain the strings 001 as substring  $\Sigma = \{0, 1\}$

$= \{001, 110000100110, 1000111001, \dots\}$

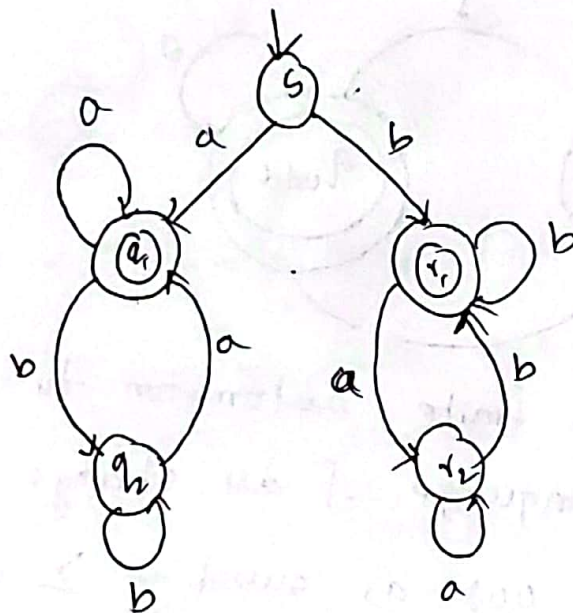
Design:



\* Design finite automata for  $L(M) = \{w \mid w \text{ is the string starts and ends with the same symbol}\}$

$\Sigma = \{a, b\}$

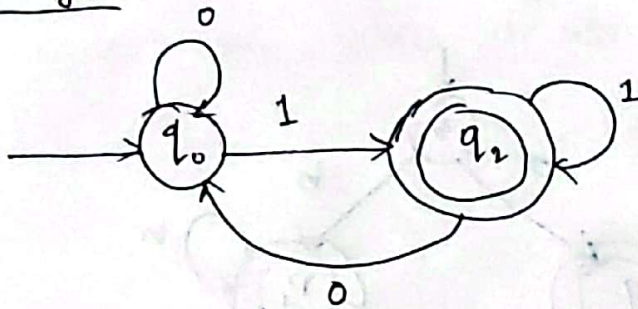
$= \{a, aa, aba, abba, abbaaba, \dots\}$   
 $\{b, bb, bab, baab, babbab, \dots\}$





\* Design finite Automaton for  $L(M) = \{w \mid w \text{ ends in a } 1\}$  and give formal description.

Design:  $= \{001, 1001, 1111, \dots\}$



Formal description

We describe finite automaton formally with

5-tuple  $(Q, \Sigma, \delta, q_0, F) =$

①  $Q = \{q_0, q_1\}$

②  $\Sigma = \{0, 1\}$

③  $\delta: Q \times \Sigma \rightarrow Q$  (SST)

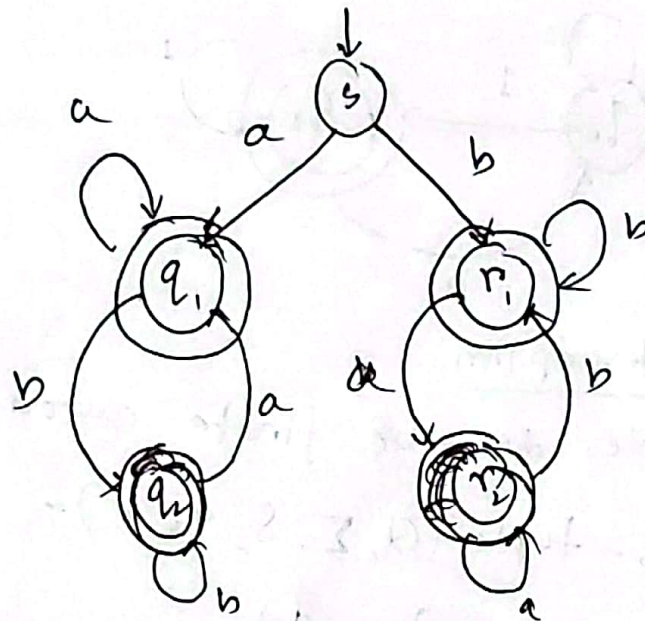
|                   |       |       |
|-------------------|-------|-------|
|                   | 0     | 1     |
| $\rightarrow q_0$ | $q_0$ | $q_1$ |
| $* q_1$           | $q_0$ | $q_1$ |

④  $q_0 = \{q_0\}$

⑤  $F = \{q_1\}$

\* Design finite Automaton for  $L(M) = \{w | w \text{ is the string starts and ends with the same symbols}\}$  and give formal description.

Design:



Formal description :

... we describe finite Automaton formally with 5-tuple  $(Q, \Sigma, \delta, q_0, F)$  -

①  $Q = \{s, q_1, q_2, r_1, r_2\}$

②  $\Sigma = \{a, b\}$

③  $\delta: Q \times \Sigma \rightarrow Q$  (SS1)

④  $q_0 = \{s\}$

⑤  $F = \{q_1, q_2, r_1, r_2\}$

|                  | a              | b              |
|------------------|----------------|----------------|
| → s              | q <sub>1</sub> | r <sub>1</sub> |
| * q <sub>1</sub> | q <sub>1</sub> | q <sub>2</sub> |
| q <sub>2</sub>   | q <sub>1</sub> | q <sub>2</sub> |
| * r <sub>1</sub> | r <sub>2</sub> | r <sub>1</sub> |
| r <sub>2</sub>   | r <sub>2</sub> | r <sub>1</sub> |

## Formal Definition of Computation

$M = (Q, \Sigma, \delta, q_0, f) \rightarrow$  finite Automaton

$w = w_1 w_2 w_3 \dots w_n \rightarrow$  string

$w_i$

$\Sigma = \{a, b\}$

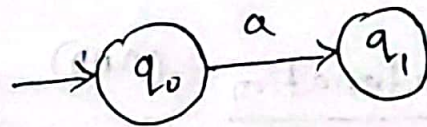
aab

aabbab

$r_0, r_1, \dots, r_n \rightarrow$  states

Three condition:

①  $r_0 = q_0$



②  $\delta(r_i, w_{i+1}) = r_{i+1}$ ; for  $i \geq 0, \dots, n-1$  and

③  $r_n \in f$



## The Regular Operation

$$A = \{ \text{good, bad} \} \rightarrow u$$

$$B = \{ \text{boy, girl} \} \rightarrow y$$

① Union

② Concatenation

③ star

Union:

$$A \cup B = \{ \text{good, bad, boy, girl} \}$$

$$A \cup B = \{ u \mid u \in A \text{ or } u \in B \}$$

Concatenation: (AND)

$$A \circ B = \{ \underbrace{\text{good}}_u \underbrace{\text{boy}}_y, \underbrace{\text{good}}_u \underbrace{\text{girl}}_y, \underbrace{\text{bad}}_u \underbrace{\text{boy}}_y, \underbrace{\text{bad}}_u \underbrace{\text{girl}}_y \}$$

$$A \circ B = \{ uy \mid u \in A \text{ and } y \in B \}$$

Star:

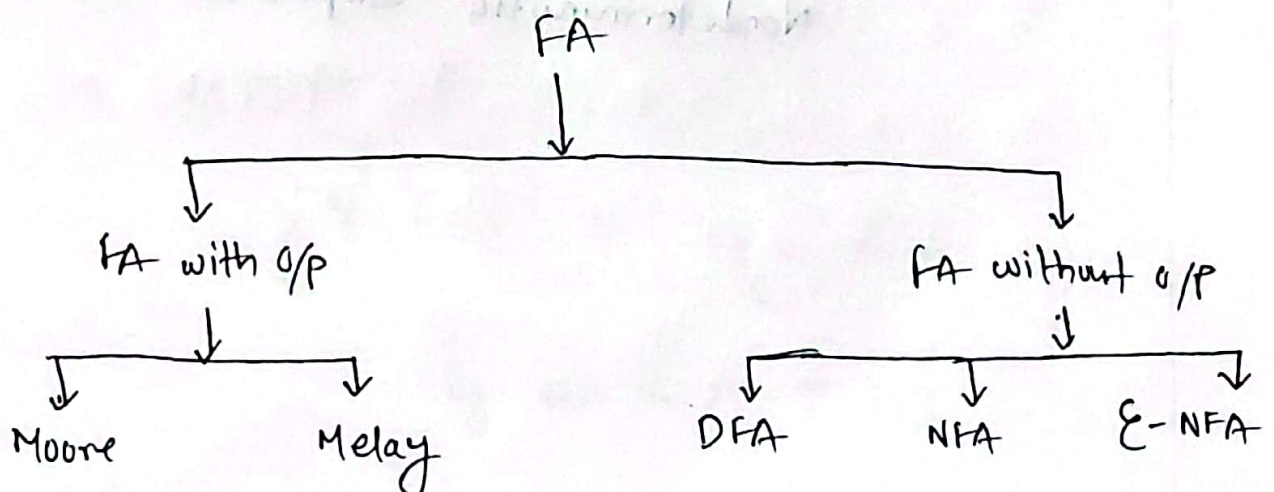
$A \rightarrow \{ \text{good, bad} \}$

$A^+ = \{ \epsilon, \text{good, bad, good good, good bad, good bad good, good good good, bad bad bad, } \dots \}$

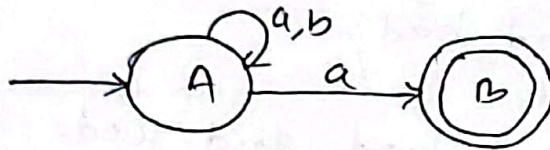
$u_1 u_2 \dots u_k \rightarrow u_i; i \geq 0$

$A^+ = \{ u_1 u_2 \dots u_k \mid i \geq 0 \text{ and each } u_i \in A \}$

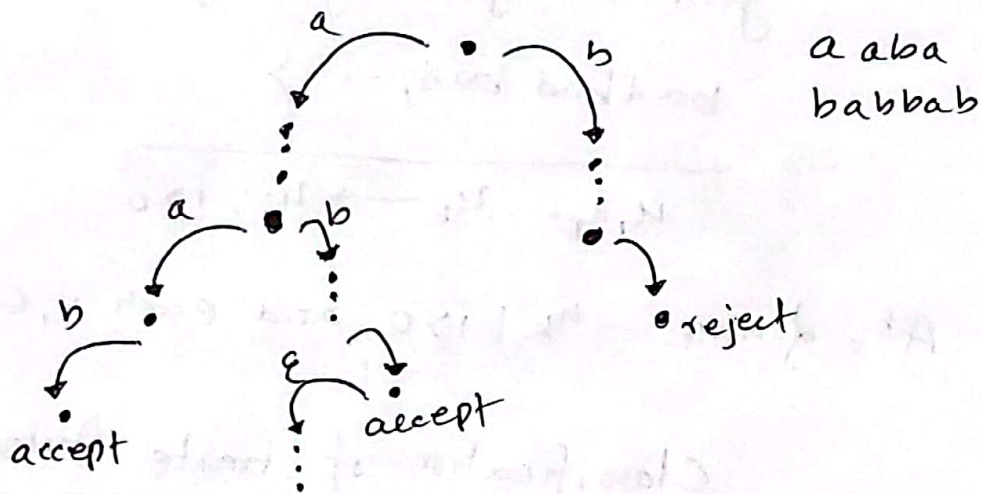
### Classification of Finite Automaton



\*  $L = \{ w \mid w \text{ ending with 'a'} \}$



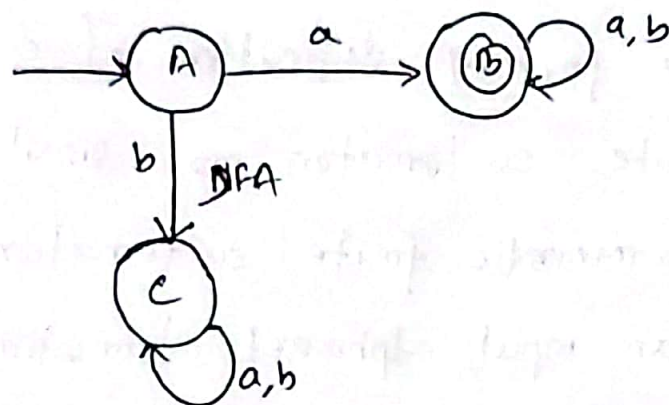
• How does an NFA compute?



Non-deterministic Computation



\*  $L = \{w \mid w \text{ starting with 'a'}\}$



start  
 $\cdot$   
 $a \hookrightarrow$   
 $\cdot$   
 $a \hookrightarrow$   
 $\cdot$   
 $b \hookrightarrow$   
 $\cdot$   
 final Accept

\* Example 1.30

Let  $A$  be the language consisting of all strings over  $\{0, 1\}$  containing a 1 in the third position from the end (e.g., 000100 is in  $A$  but 000 is not). The following four-state NFA  $N_2$  recognizes  $A$ .

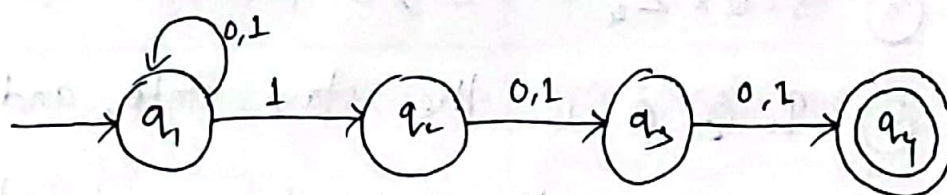


Fig: NFA  $N_2$  recognizing  $A$ .

#### \* Formal Definition of NFA

The formal definition of a nondeterministic finite automaton is similar to that of a deterministic finite automaton. Both have states, an input alphabet, transition function, start state and a collection of accept states.

A nondeterministic finite Automaton is a 5-tuple  $(Q, \Sigma, \delta, q_0, F)$ , where

- (1)  $Q$  is a finite set of states
- (2)  $\Sigma$  is a finite alphabet
- (3)  $\delta: Q \times \Sigma \rightarrow P(Q)$  is the transition function
- (4)  $q_0 \in Q$  is the start state, and
- (5)  $F \subseteq Q$  is the set of accept states.

## \* Equivalence of NFAs AND DFAs

Deterministic and non

Deterministic and nondeterministic finite automata recognize the same class of languages. Such equivalence is both surprising and useful.

It is surprising because NFAs appear to have more power than DFAs, so we might expect that NFAs recognize more languages. It is useful because describing an NFA for a given language sometimes is much easier than describing a DFA for that language.

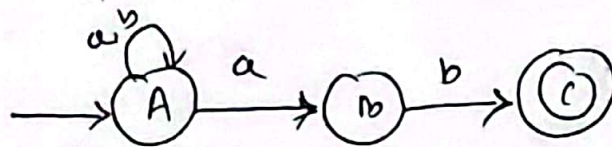
| NFA                                               | DFA                                                       |
|---------------------------------------------------|-----------------------------------------------------------|
| NFA stands for non-deterministic finite automata. | DFA stands for deterministic finite automata              |
| Backtracking is not allowed in NFA.               | Backtracking is allowed in DFA                            |
| Easy to design due to non-deterministic           | More difficult to design due to deterministic transition. |
| string is <del>not</del>                          |                                                           |



| NFA                                                                              | DFA                                                             |
|----------------------------------------------------------------------------------|-----------------------------------------------------------------|
| It permits empty string transition                                               | It cannot permit empty string transition.                       |
| NFA requires less memory space.                                                  | DFA requires more memory space.                                 |
| String is accepted by NFA if one of all possible transition ends in final state. | String is accepted by DFA if the transition D into final state. |

# NFA to DFA Conversion (Subset Construction)

$L = \{ w \mid w \text{ ends with } ab \}$



NFA

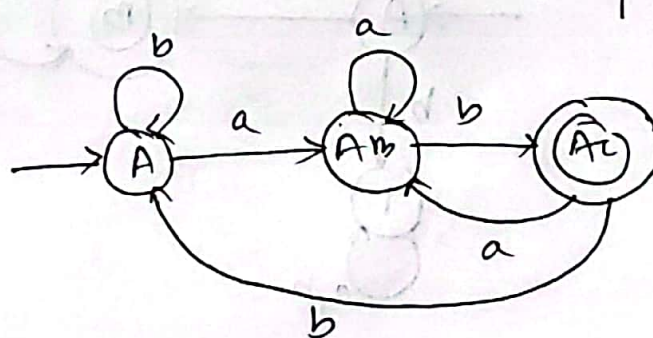
STT for NFA

|     | a           | b           |
|-----|-------------|-------------|
| → A | $\{A, B\}$  | $\{A\}$     |
| B   | $\emptyset$ | $\{C\}$     |
| *C  | $\emptyset$ | $\emptyset$ |

STT for DFA

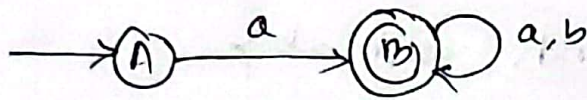
|       | a    | b    |
|-------|------|------|
| → [A] | [AB] | [A]  |
| [AB]  | [AB] | [AC] |
| *[AC] | [AB] | [A]  |

$\{A, B\} \cup \emptyset$   
 $= \{A, B\}$   
 $\{A\} \cup \{C\}$   
 $= \{A, C\}$



DFA

\*  $L = \{ w \mid w \text{ starting with 'a'} \}$



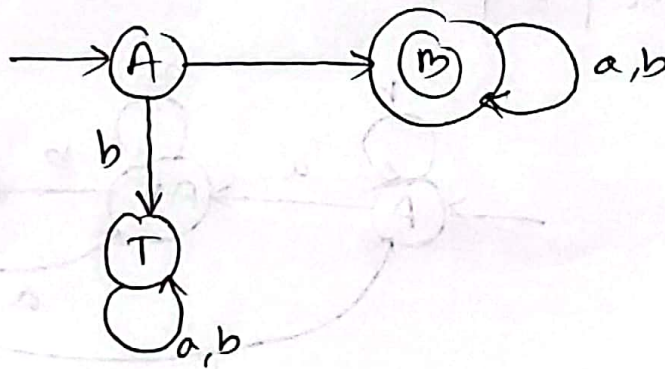
NFA

SIT for NFA

|     | a   | b                 |
|-----|-----|-------------------|
| → A | {B} | ∅ → Death Config. |
| * B | {B} | {B}               |

SIT for DFA

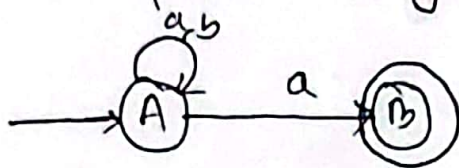
|       | a   | b   |
|-------|-----|-----|
| → [A] | [B] | [T] |
| * [B] | [B] | [B] |
| [T]   | [T] | [T] |



DFA



\*  $L = \{w \mid w \text{ ending with 'a'}\}$



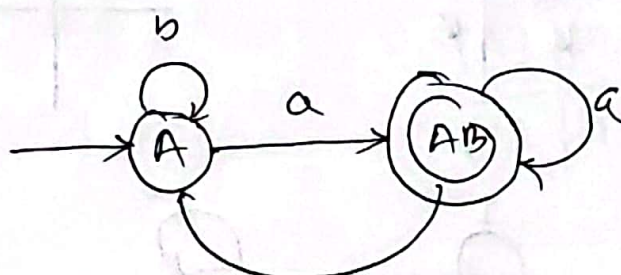
NFA

SIT for NFA

|     | a      | b   |
|-----|--------|-----|
| → A | {A, B} | {A} |
| * B | ∅      | ∅   |

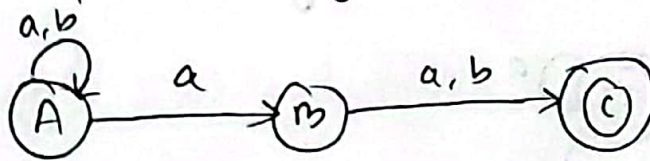
SIT for DFA

|        | a    | b   |
|--------|------|-----|
| → [A]  | [AB] | [A] |
| * [AB] | [AB] | [A] |



DFA

\* 1<sup>st</sup> & 2<sup>nd</sup> symbol from RHS is 'a'

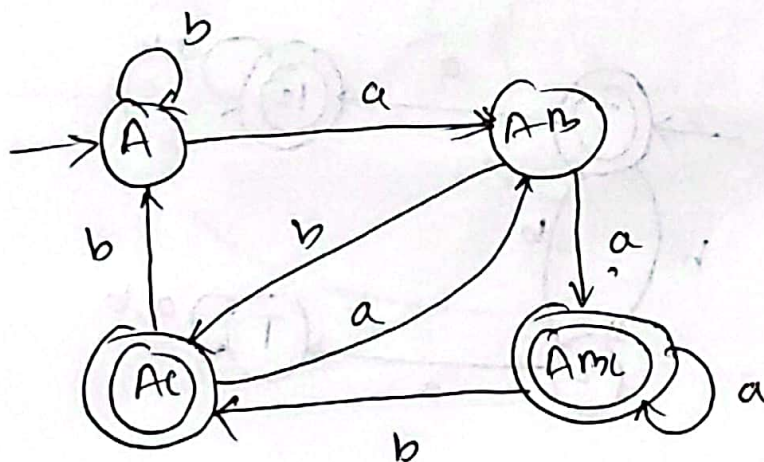


SIT for NFA

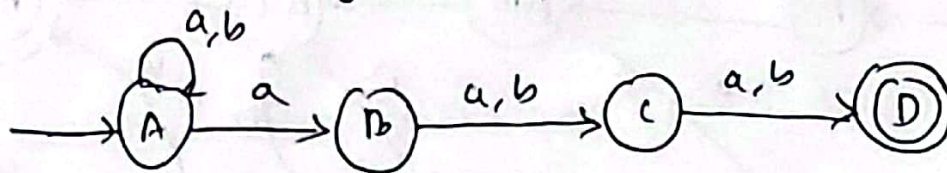
|     | a      | b   |
|-----|--------|-----|
| → A | {A, B} | {A} |
| B   | {C}    | {C} |
| * C | ∅      | ∅   |

SIT for DFA

|         | a     | b    |
|---------|-------|------|
| → [A]   | [AB]  | [A]  |
| [AB]    | [AB]  | [AC] |
| * [AC]  | [AC]  | [A]  |
| * [ABC] | [ABC] | [AC] |



\* 1<sup>st</sup> 2<sup>nd</sup> 3<sup>rd</sup> symbol from RHS is 'a'



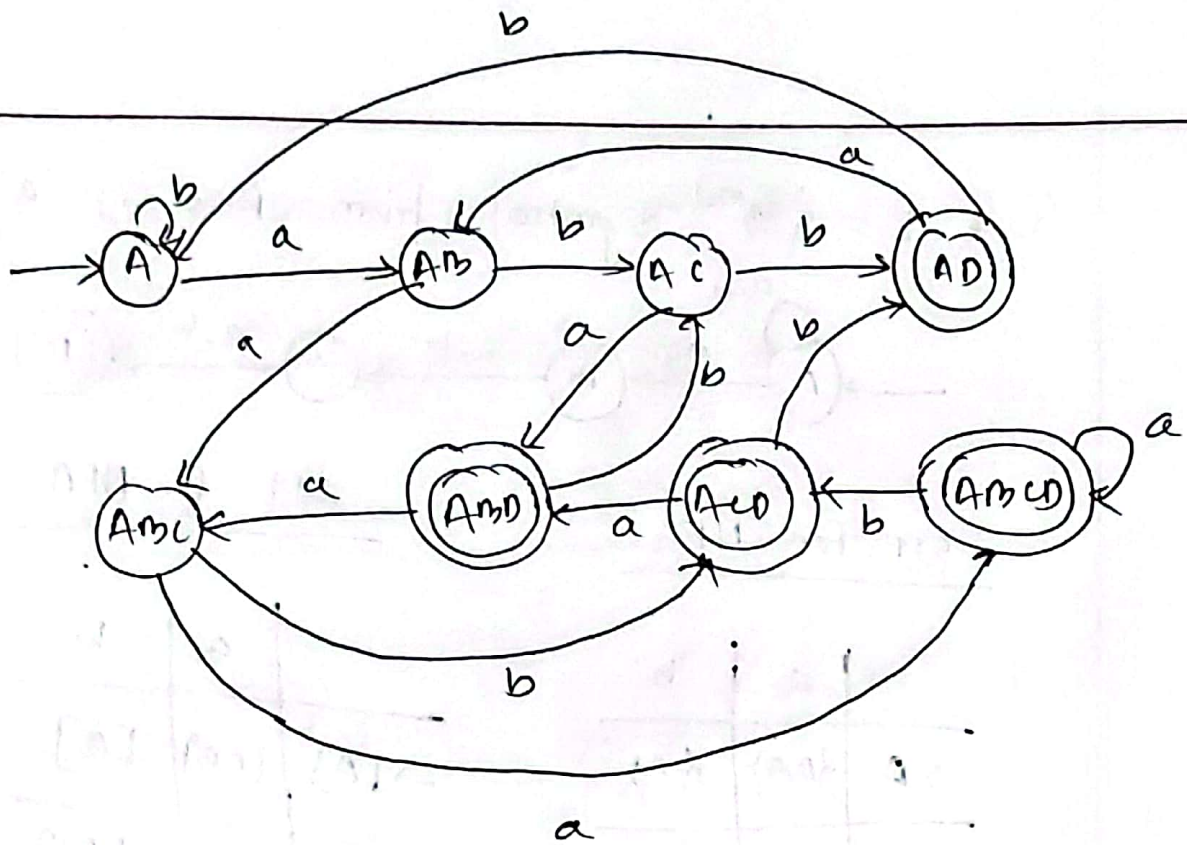
STF for NFA

|     | a      | b   |
|-----|--------|-----|
| → A | {A, B} | {A} |
| B   | {C}    | {C} |
| C   | {D}    | {D} |
| * D | ∅      | ∅   |

SIT for DFA

|                | a         | b         |
|----------------|-----------|-----------|
| → [A]          | [A, B]    | [A]       |
| [A, B]         | [A, B, C] | [A, C]    |
| [A, C]         | [A, B, D] | [A, D]    |
| * [A, D]       | [A, B]    | [A]       |
| [A, B, C]      | [A, B, D] | [A, C, D] |
| * [A, B, D]    | [A, B, C] | [A, C]    |
| * [A, C, D]    | [A, B, D] | [A, D]    |
| * [A, B, C, D] | [A, B, D] | [A, C, D] |

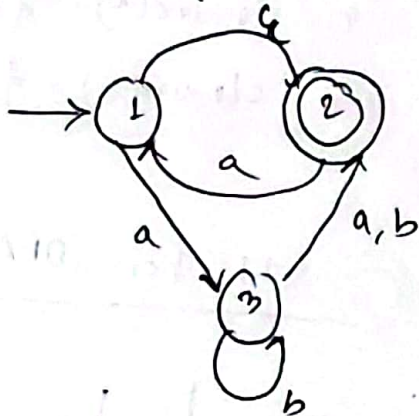




DFA

## $\epsilon$ -NFA to DFA

\* Convert the following  $\epsilon$ -NFA to DFA



$$\epsilon\text{-closure}(1) = \{1, 2\}$$

$$\epsilon\text{-closure}(2) = \{2\}$$

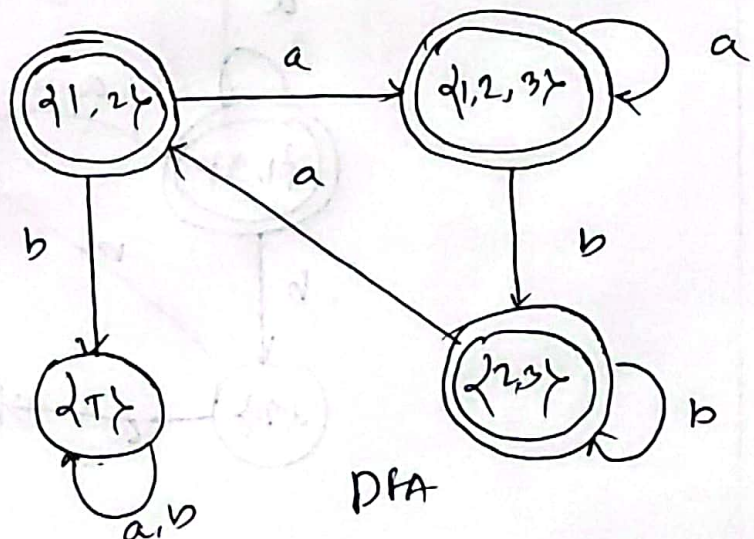
$$\epsilon\text{-closure}(3) = \{3\}$$

STT for NFA

|                     | a       | b           |
|---------------------|---------|-------------|
| $\rightarrow \{1\}$ | $\{3\}$ | $\emptyset$ |
| $\ast \{2\}$        | $\{1\}$ | $\emptyset$ |
| $\{3\}$             | $\{2\}$ | $\{2, 3\}$  |

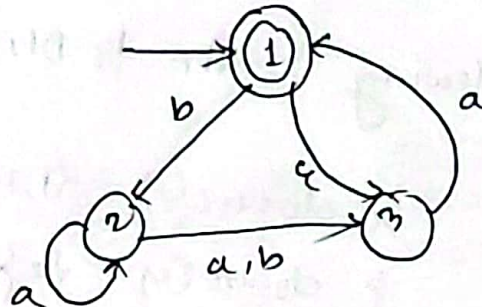
STT for DFA

|                        | a             | b          |
|------------------------|---------------|------------|
| $\rightarrow \{1, 2\}$ | $\{1, 2, 3\}$ | $\{1\}$    |
| $\ast \{1, 2, 3\}$     | $\{1, 2, 3\}$ | $\{2, 3\}$ |
| $\ast \{2, 3\}$        | $\{1, 2\}$    | $\{2, 3\}$ |
| $\{1\}$                | $\{1\}$       | $\{1\}$    |



DFA

\* Convert the following  $\epsilon$ -NFA to DFA



$$\epsilon\text{-closure}(1) = \{1, 3\}$$

$$\epsilon\text{-closure}(2) = \{2\}$$

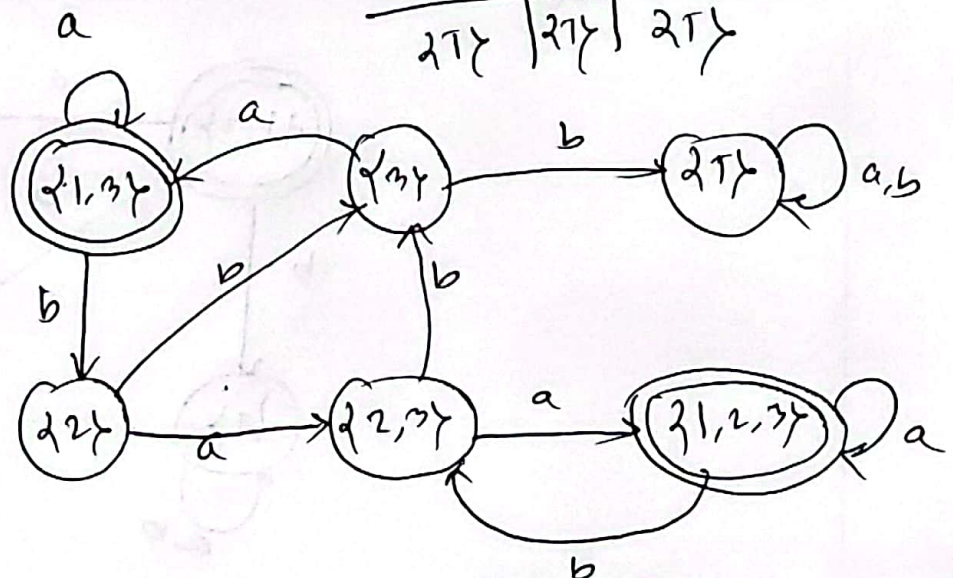
$$\epsilon\text{-closure}(3) = \{3\}$$

STT for NFA

|                     | a           | b           |
|---------------------|-------------|-------------|
| $\rightarrow \{1\}$ | $\emptyset$ | $\{2, 3\}$  |
| $\{2\}$             | $\{2, 3\}$  | $\{3\}$     |
| $\{3\}$             | $\{1\}$     | $\emptyset$ |

STT for DFA

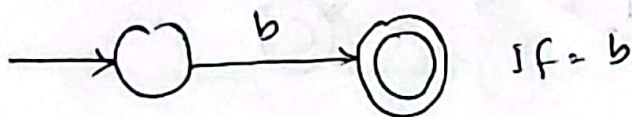
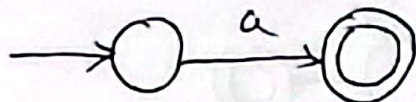
|                        | a             | b          |
|------------------------|---------------|------------|
| $\rightarrow \{1, 3\}$ | $\{1, 3\}$    | $\{2\}$    |
| $\{2\}$                | $\{2, 3\}$    | $\{3\}$    |
| $\{3\}$                | $\{1, 3\}$    | $\{1\}$    |
| $\{1, 2, 3\}$          | $\{1, 2, 3\}$ | $\{3\}$    |
| $\{1, 2, 3\}$          | $\{1, 2, 3\}$ | $\{2, 3\}$ |
| $\{2, 3\}$             | $\{2, 3\}$    | $\{2, 3\}$ |





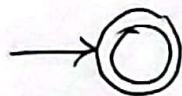
## Formal Def<sup>n</sup> of Regular Expression

1.  $R = a ; a \in \Sigma$

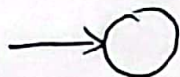


If = b

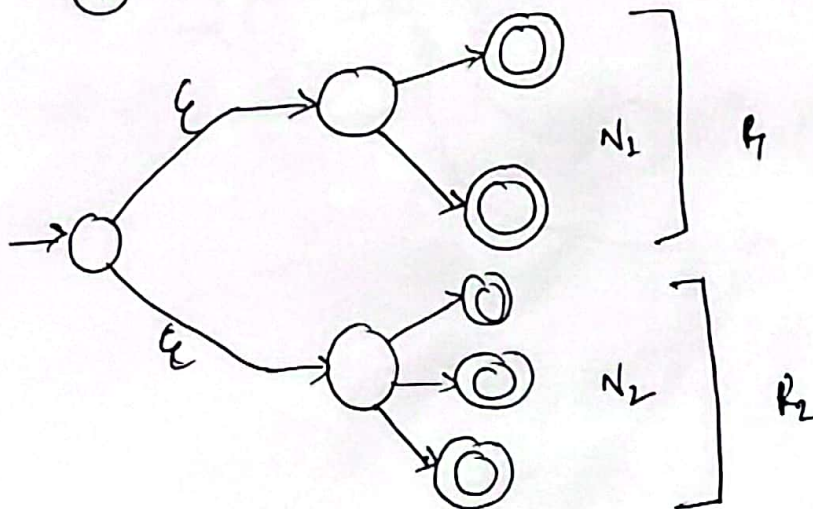
②  $R = \epsilon$ ; Initial state is the final state.



③  $R = \emptyset$

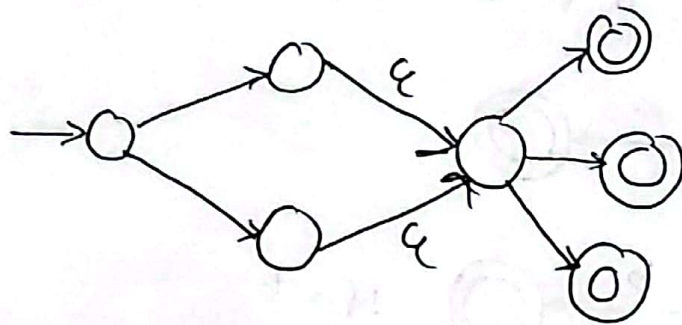


④  $R = R_1 \cup R_2$  or,  $R_1 | R_2$

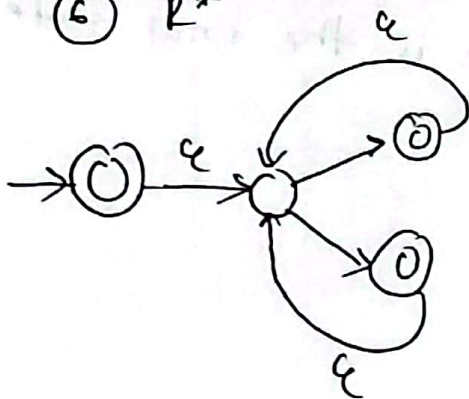


(5)  $R = R_1 \cup R_2$  or  $R_1 R_2$

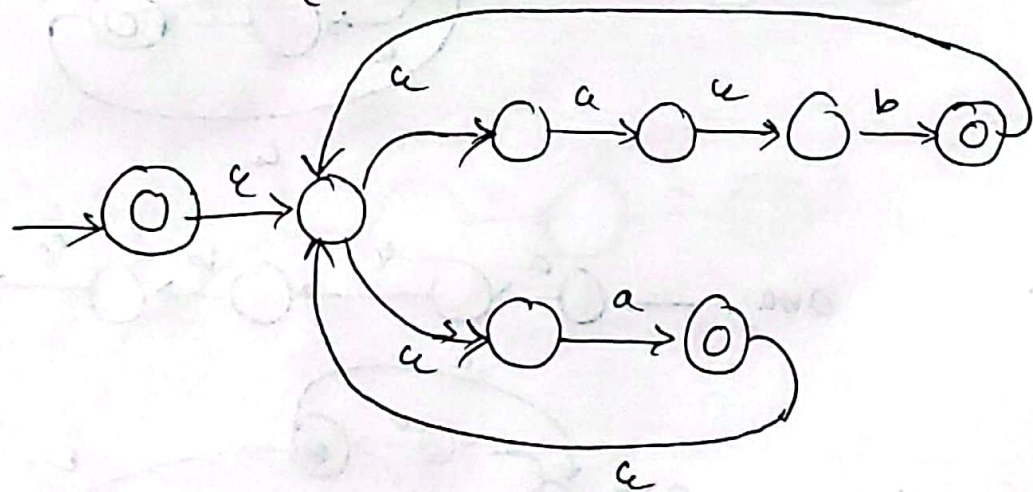
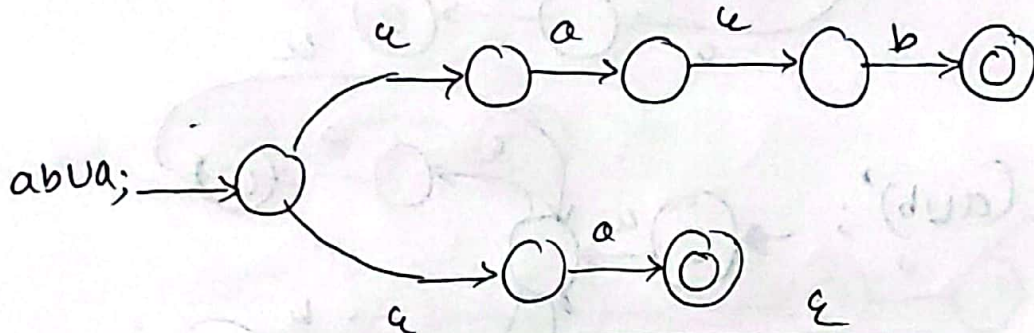
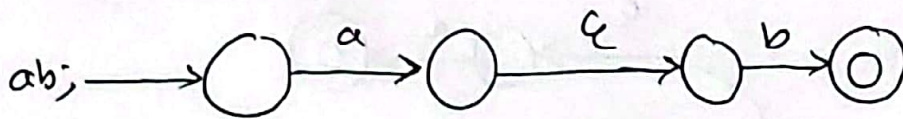
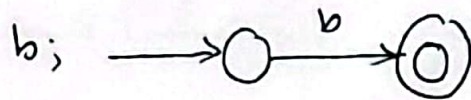
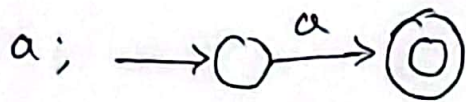
Previous NFA



(6)  $R^*$



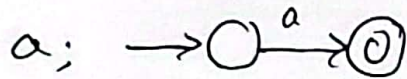
\* Draw NFA for the regular expression  $(ab \cup a)^+$



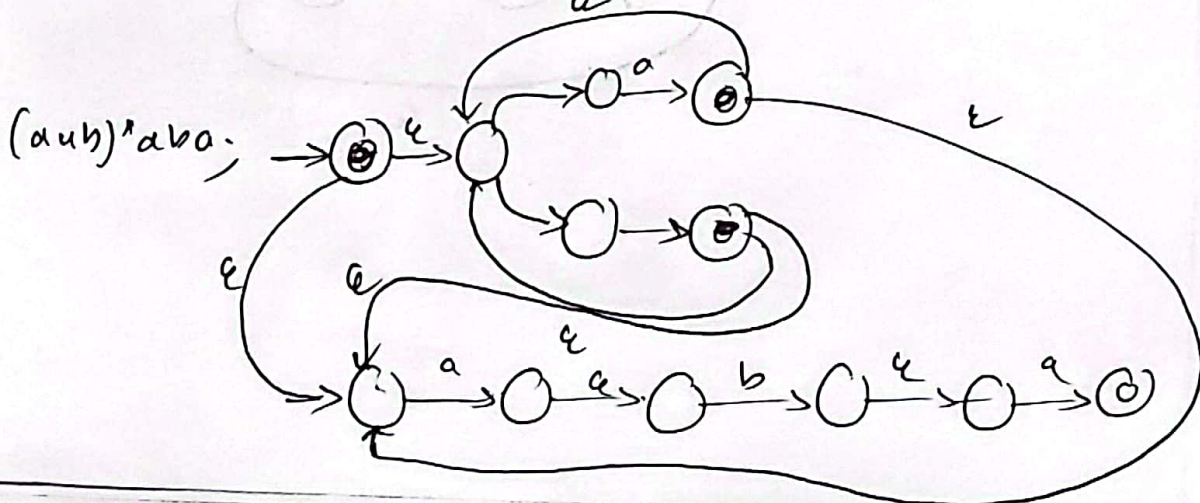
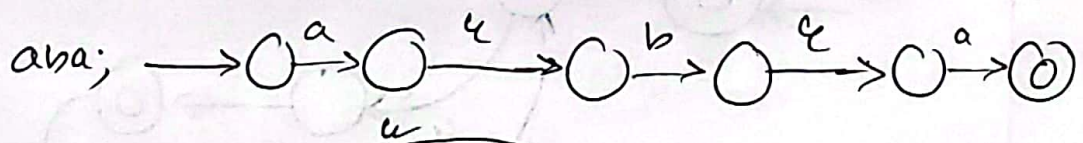
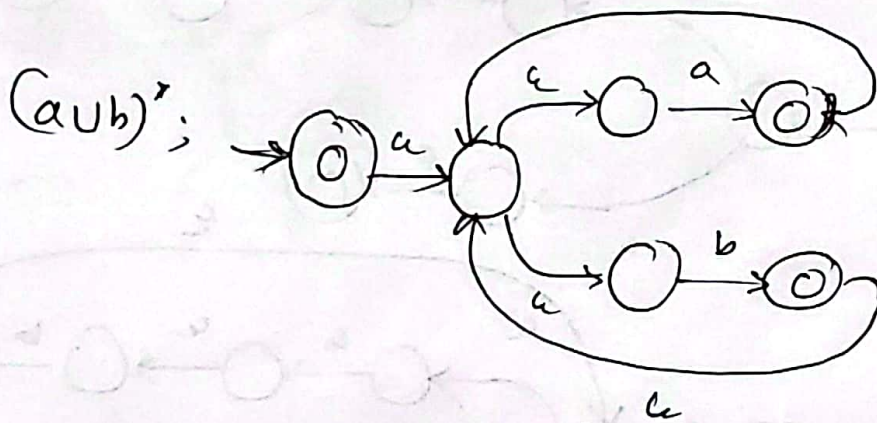
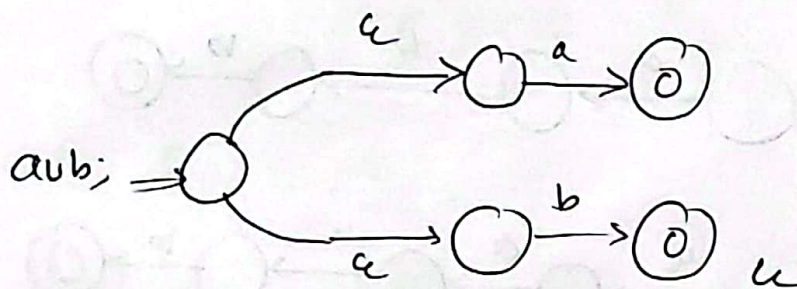
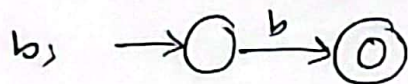


\* Draw NFA for the regular expression  $(a|b)^*aba$ .

∴



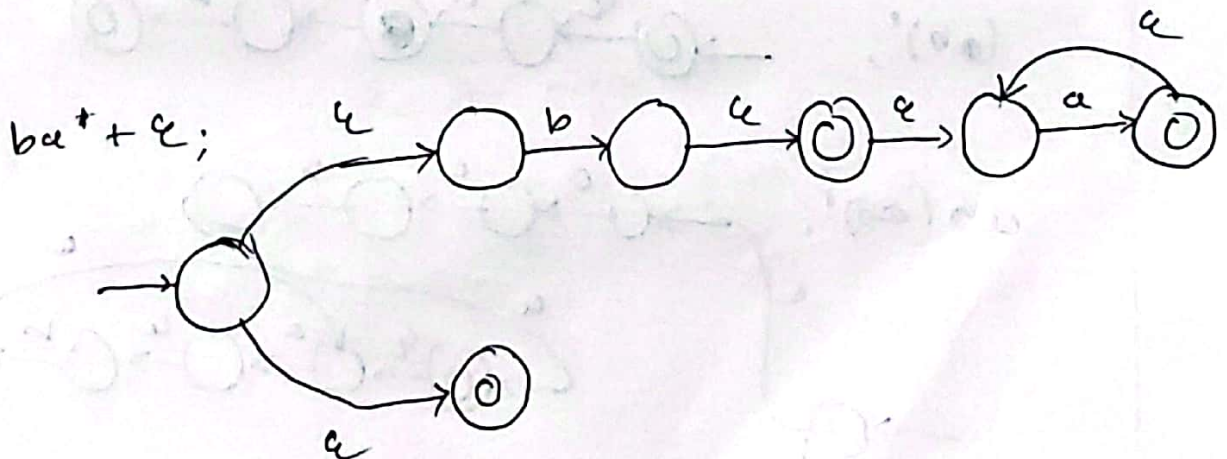
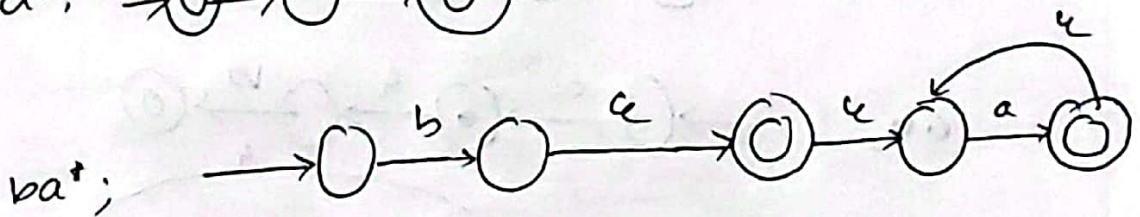
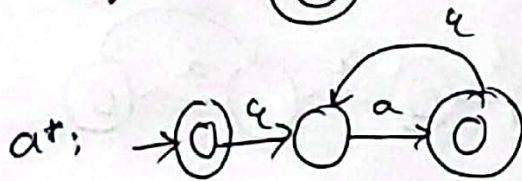
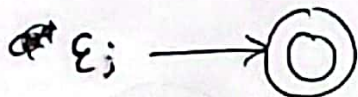
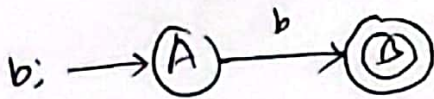
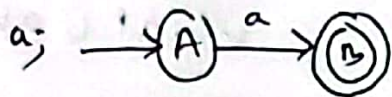
(31)



\* Draw NFA for the regular expression  $ba^+ + \epsilon$ .

$(ba^+ \cup \epsilon)$   
like

2)





Draw NFA for the regular expression

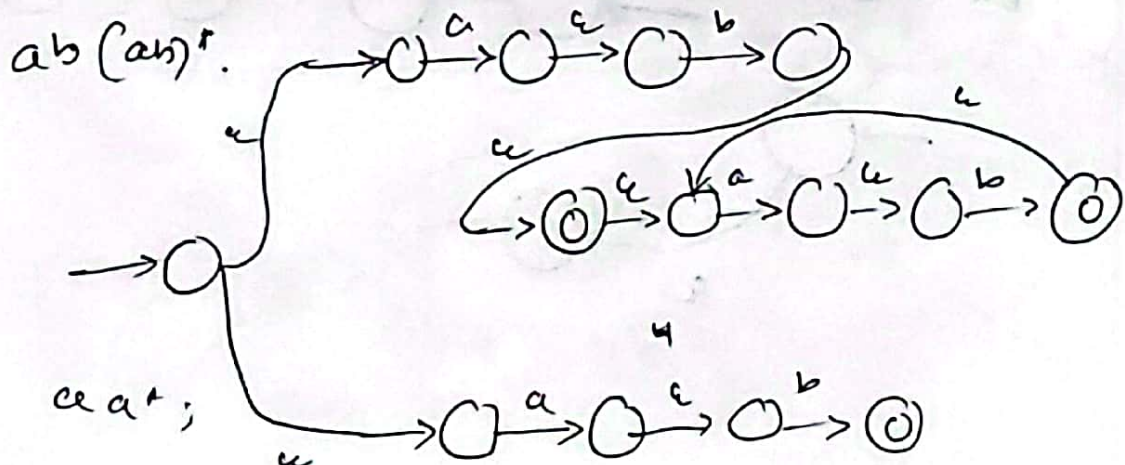
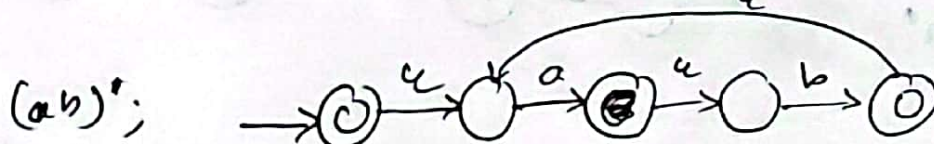
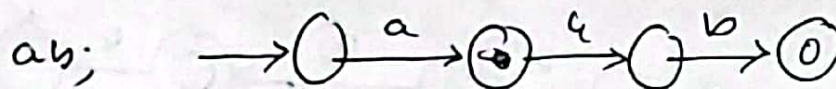
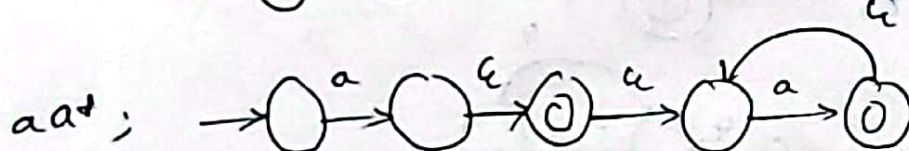
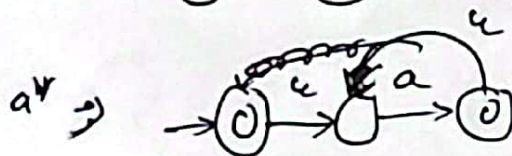
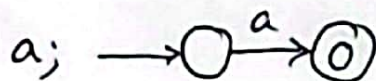
$$a^+ \cup (ab)^+$$

$$a^+ = aa^+$$

$$(ab)^+ = ab(ab)^+$$

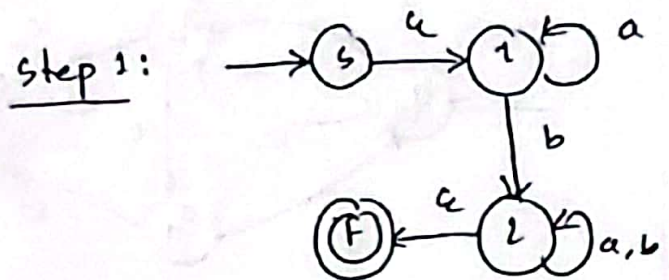
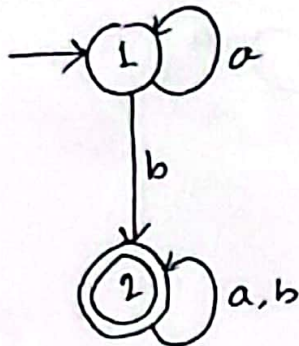
$$\checkmark aa^+ \cup ab(ab)^+$$

→



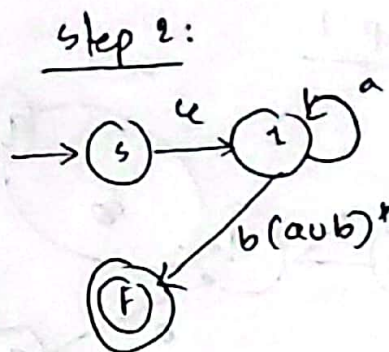


\* Convert the NFA into a regular Expression.



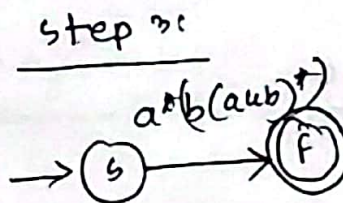
Rip 2

| In | Out |
|----|-----|
| 1  | F   |



Rip 1

| In | Out |
|----|-----|
| s  | F   |

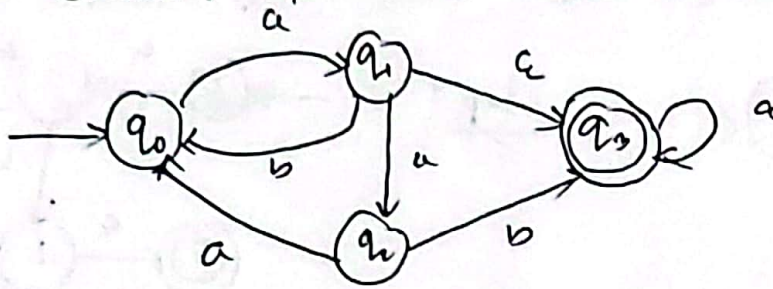


So, the regex is:

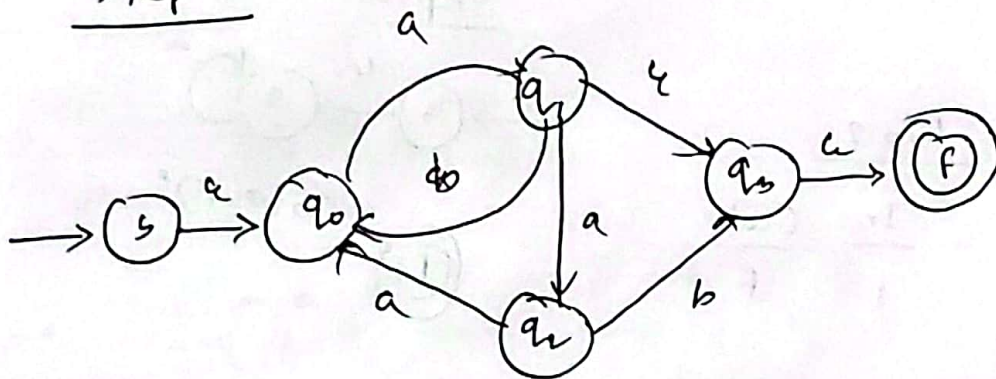
$a^*(b(aub)^*)$

Ans

\* Convert the NFA into a regular expression.



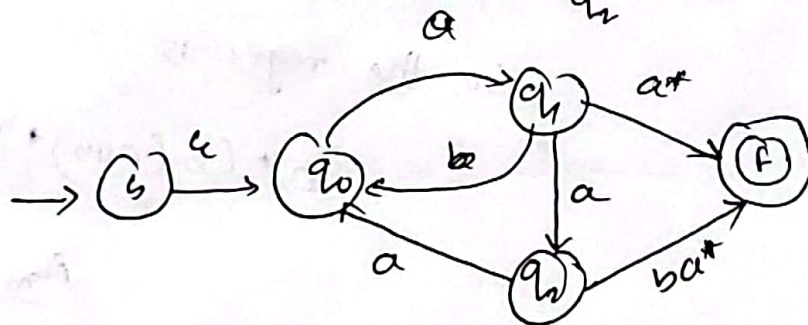
= Step:1



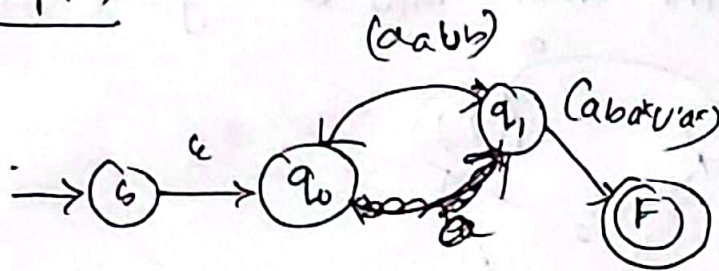
Step:2

rip  $q_3$

| In    | out |
|-------|-----|
| $q_1$ | F   |
| $q_2$ |     |



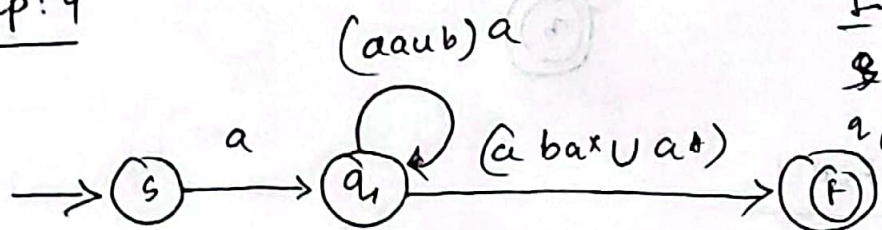
step: 3



| RIP |         |
|-----|---------|
| In  | Out     |
| q1  | q0<br>F |

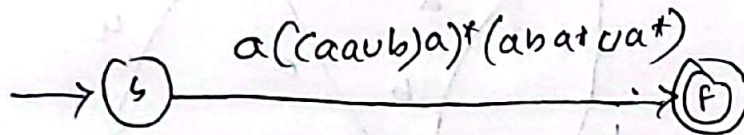
$a b a^+ u a^+$

step: 4



| RIP |         |
|-----|---------|
| In  | Out     |
| q1  | q1<br>F |

step: 5



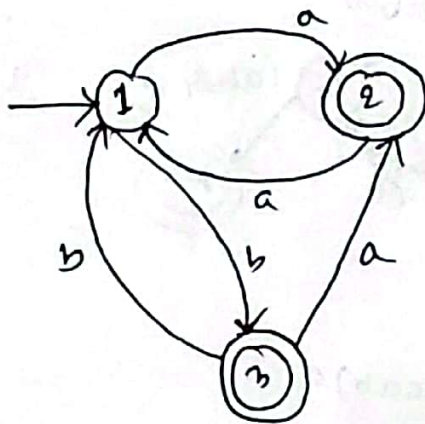
| RIP |     |
|-----|-----|
| In  | Out |
| S   | F   |

Reg ex:

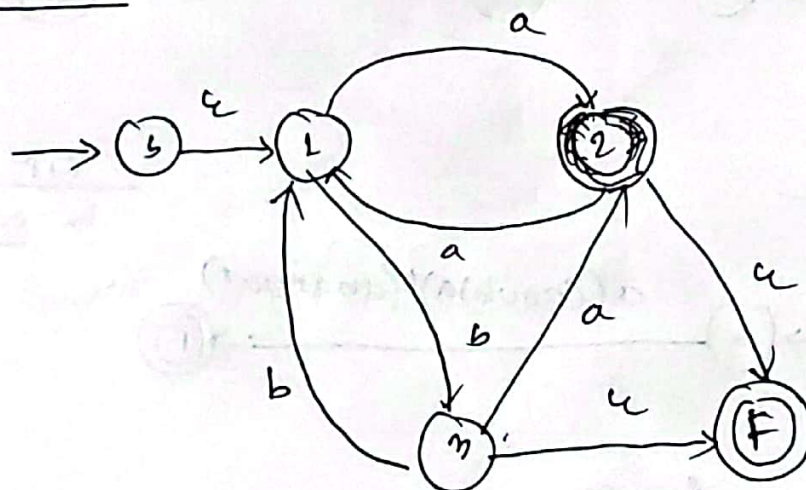
$a((a a u b) a)^+ (a b a^+ u a^+)$



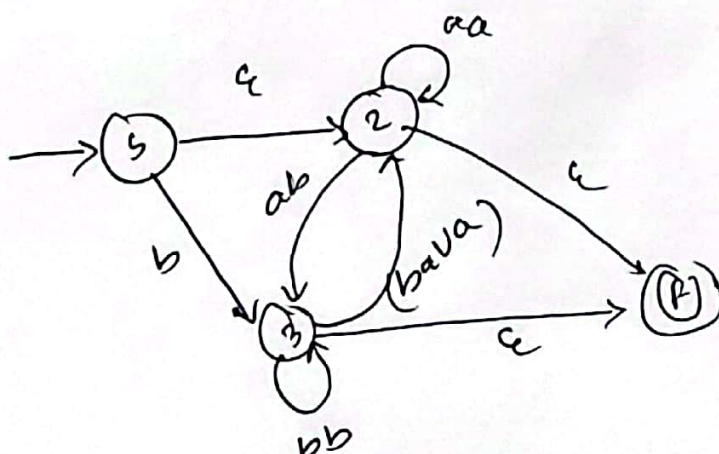
\* Convert the NFA into a regular Expression.



Step: 1

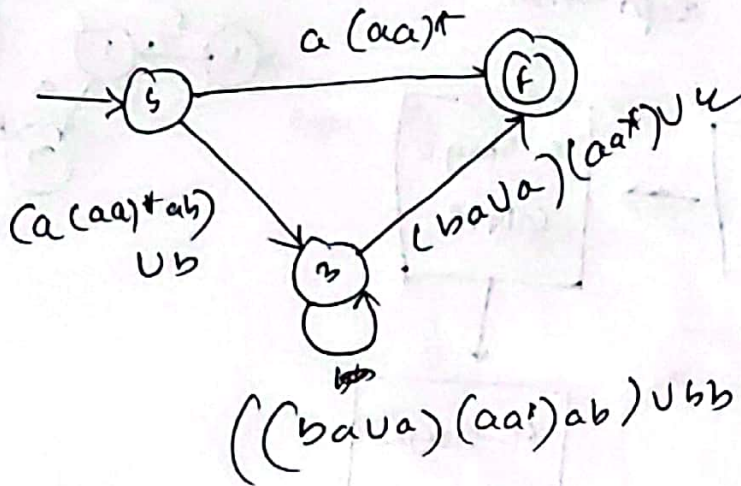


Step: 2



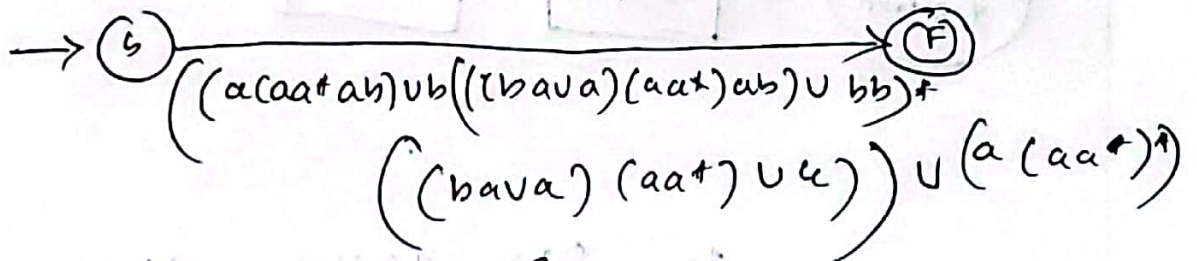
| R/P |   |
|-----|---|
| 1   | 2 |
| 2   | 3 |
| 3   | 2 |

Step: 3



| MD |     |
|----|-----|
| In | out |
| 5  | 3   |
| 3  | F   |

Step: 4



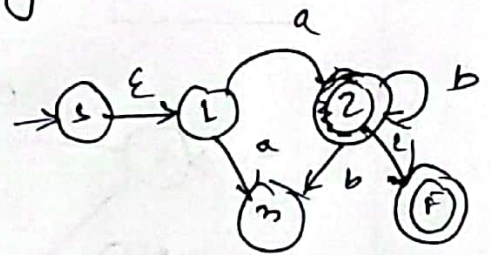
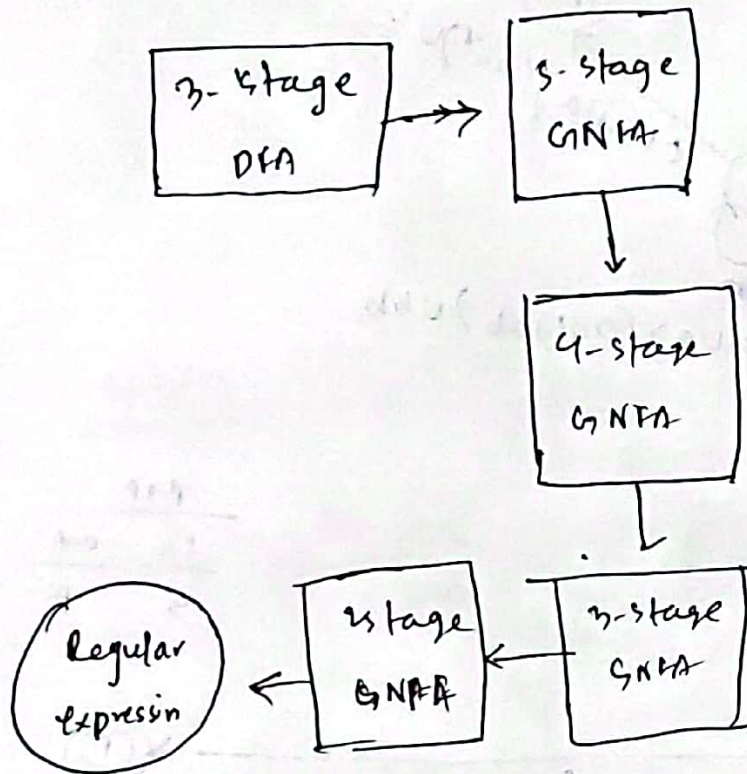
| MD |     |
|----|-----|
| In | out |
| 5  | F   |

Regex is:

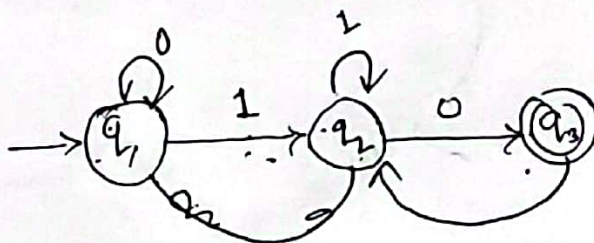
Ans:



\* Typical stages in converting a NFA/DFA to a regular expression:



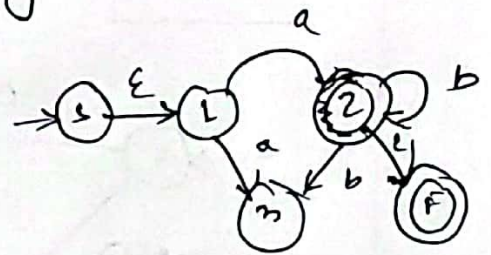
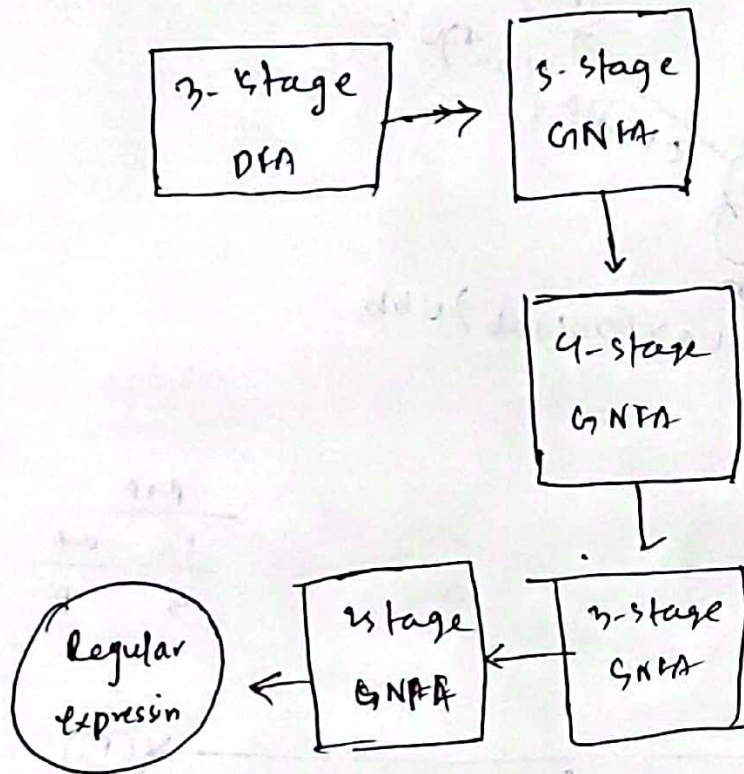
10, 010, 1010, 1101



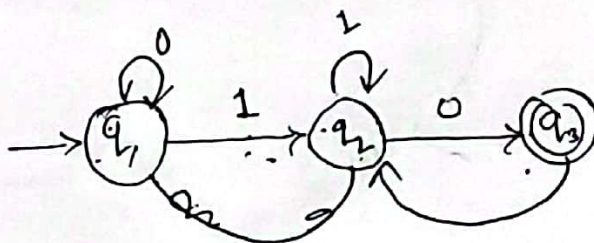
0101  
0100  
1  
0110



\* Typical stages in converting a NFA/DFA to a regular expression:



10, 010, 1010, 1101



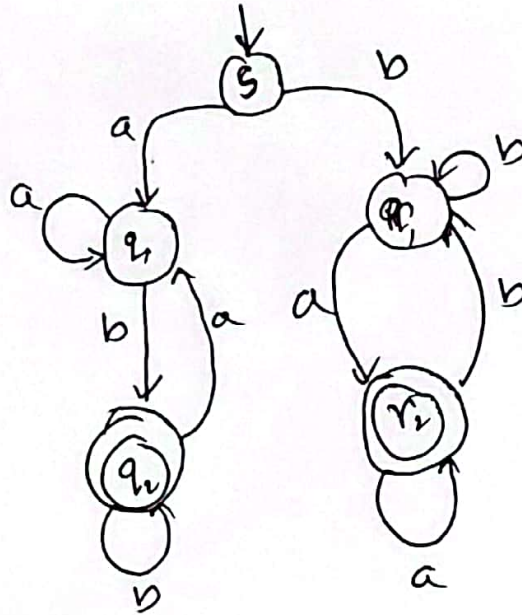
0101  
0100  
1  
0110

Spring 2024

1(b)

start to end diff symbol

i)



ii)

# Theory of Computing

Complexity Theory

easy / Hard

Sorting  
Ascending

Scheduling

No two classes  
take place  
at the same  
place at the  
same time.

Computability Theory

Solvable

Not solvable

Automata Theory

Mathematical

Model  $\rightarrow$  def &

properties

$\rightarrow$  Text processing

① Finite Automata

$\rightarrow$  Compiler

$\rightarrow$  Hardware design

② Context-free

Grammar  $\rightarrow$

Programming language

Artificial intelligence



## Theory of Computing

Complexity Theory

easy / Hard

Sorting  
Ascending

Scheduling

No two classes  
take place  
at the same  
place at the  
same time.

Computability Theory

Solvable

Not solvable

Automata Theory

Mathematical

Model → def &

properties

→ Text processing

① Finite Automata

→ Compiler

→ Hardware design

② Context-free

Grammar →

Programming language

Artificial intelligence

## Set theory

Set: A set is a group of objects represented as a unit. Sets may contain any type of object including numbers, symbols and even other sets.

Example:

$$S_1 = \{1, 2, 3, 4\} \quad S_2 = \{a, b, c, d\}$$

$$S_3 = \{a, b, c, 1, 2, 3\} \quad S_4 = \{\{a, b, c\}, \{1, 2, 3\}\}$$

## Infinite sets

"..."  $\rightarrow$  Continue the sequence forever.

① set of natural numbers.

$$N = \{1, 2, 3, 4, 5, \dots\}$$

② set of Integer.

$$Z = \{\dots, -3, -2, -1, 0, 1, 2, 3, \dots\}$$



Empty set:  $S_1 = \{\}$ ,  $S_2 = \emptyset$

Singleton set:  $S_1 = \{1\}$ ,  $S_2 = \{a\}$

Unordered pair:  $S_1 = \{1, 2\}$

Subset: For two sets  $A$  and  $B$ , we say that  $A$  is a subset of  $B$ , written  $A \subseteq B$ , if every member of  $A$  also is a member of  $B$ .

$$B = \{1, 2, 3, a, b, c\}$$

$$A = \{1, 2, 3\}$$

$$A \subseteq B$$

Proper Subset: For two sets  $A$  and  $B$ , we say that  $A$  is a proper subset of  $B$ , written  $A \subset B$ , if  $A$  is a subset of  $B$  and not equal to  $B$ .

$$B = \{1, 2, 3, a\}$$

$$A = \{1, 2\}$$

$$A \subset B$$



Empty set:  $S_1 = \{\}$ ,  $S_2 = \emptyset$

Singleton set:  $S_1 = \{1\}$ ,  $S_2 = \{a\}$

Unordered pairs:  $S_1 = \{1, 2\}$

Subset: For two sets  $A$  and  $B$ , we say that  $A$  is a subset of  $B$ , written  $A \subseteq B$ , if every member of  $A$  also is a member of  $B$ .

$$B = \{1, 2, 3, a, b, c\}$$

$$A = \{1, 2, 3\}$$

$$A \subseteq B$$

Proper Subset: For two sets  $A$  and  $B$ , we say that  $A$  is a proper subset of  $B$ , written  $A \subset B$ , if  $A$  is subset of  $B$  and not equal to  $B$ .

$$B = \{1, 2, 3, a\}$$

$$A = \{1, 2\}$$

$$A \subset B$$